

ThunderERP

The Complete NestJS Intern Reference

Intellectual Property of AB Global Traders

*This is your long-term technical reference — not a one-time read.
You will not understand everything immediately. That is perfectly expected.
Come back to this document at every stage of your career.*

0. How To Use This Document

Three Layers of Learning

Layer 1 — Week 1: Understand the request flow (Controller to Service to Database). Build your first CRUD endpoint.

Layer 2 — Week 2-3: Master DTOs, validation, Prisma ORM, relationships, and error handling.

Layer 3 — Week 4+: Transactions, state machines, concurrency, authentication, audit logging, and performance.

This document is intentionally comprehensive and deep. You are not expected to master everything at once. Use it as a reference — when you encounter a problem or concept, find the relevant section, read it carefully, and implement it.

WARNING: Do Not Memorise. Understand.

The goal is system thinking — understanding WHY things are structured the way they are. Memorising syntax is useless; understanding data flow makes you a senior engineer. Every rule in this document exists for a real reason. Ask "why" constantly. Senior engineers are not those who know the most syntax. They ask the best questions.

1. System Overview & Architecture

1.1 What is ThunderERP?

ThunderERP is a full-stack Enterprise Resource Planning (ERP) system. An ERP manages all core business operations — inventory, sales, finance, HR, and CRM — in one unified system. Losing data integrity in an ERP does not just crash a website; it corrupts financial records and costs real money.

ThunderERP uses a Modular Monolith architecture: all code lives in one repository, but every business feature (Inventory, Orders, CRM, Finance) is strictly separated into its own NestJS Module. This gives us the organisation of microservices without the operational complexity of running dozens of independent services.

1.2 The Three-Tier Architecture

Every web application built today runs on three tiers. Understanding this separation is the single most important architectural concept you will learn:

Tier	Technology	Responsibility	What It Must NEVER Do
Frontend (Client)	React	Capture user intent, display data, handle forms and UI state	Make business decisions, calculate

Tier	Technology	Responsibility	What It Must NEVER Do
			totals, check permissions, trust itself
Backend (Server)	NestJS on Node.js	Validate all input, run business logic, enforce security rules, orchestrate data	Render HTML for display, store large files locally, trust the frontend
Database	PostgreSQL + Prisma ORM	Store data persistently, enforce referential integrity, handle concurrent writes	Run business logic, expose itself directly to the internet, be accessed without ORM

1.3 The Request Lifecycle — The Golden Path

Every single user action in ThunderERP follows this exact path. When debugging ANY bug, trace it step by step through this chain. The bug is always at one of these handoff points:

THE GOLDEN PATH — REQUEST LIFECYCLE

```
User clicks "Create Order" button on the React UI
|
v
React sends: POST https://api.thundererp.com/orders
    Headers: { Authorization: "Bearer eyJ..." }
    Body:    { customerId: 5, items: [...] }
|
v
NestJS JwtAuthGuard checks: Is this token valid? Is it expired?
NestJS RolesGuard checks:  Does this user have permission to create orders?
|
v
NestJS Controller receives request:
- ValidationPipe validates DTO shape (are required fields present? are types correct?)
- Calls this.ordersService.create(dto, userId)
|
v
NestJS Service runs business logic:
- Checks if customer exists
- Checks if all products are in stock
- Calculates order total
- Validates business rules (minimum order value, etc.)
|
v
Prisma ORM translates TypeScript into SQL:
INSERT INTO orders (customer_id, total, status) VALUES (5, 150.00, PENDING);
```

```
INSERT INTO order_items (...) VALUES (...);
UPDATE products SET quantity = quantity - 2 WHERE id = 12;
|
v
PostgreSQL executes SQL atomically, returns affected rows
|
v
Service returns the newly created order object
|
v
Controller sends HTTP 201 Created with the order JSON
|
v
React receives response, updates UI, shows success notification
```

Key Insight

If you can trace ANY feature through this path, you understand backend development. Every bug you will ever encounter is a break somewhere in this exact chain. Always ask: "At which step did this go wrong?" This is how experts debug.

2. Environment Setup (Do This First)

2.1 Required Tools — Install in This Order

Tool	What It Does	How to Install	Verify Installation
Node.js LTS	JavaScript runtime that powers NestJS	Download from nodejs.org — choose the LTS version	<code>node --version</code>
npm	Package manager for installing libraries	Automatically included with Node.js	<code>npm --version</code>
NestJS CLI	Command-line tool for generating NestJS files	<code>npm install -g @nestjs/cli</code>	<code>nest --version</code>
Docker Desktop	Runs PostgreSQL in a container so you do not install it directly	Download from docker.com/get-started	<code>docker --version</code>
Postman	GUI tool for testing API endpoints manually	Download from postman.com	Open the application
VS Code	Code editor with best NestJS support	Download from code.visualstudio.com	Open any .ts file

Tool	What It Does	How to Install	Verify Installation
Git	Version control — track changes, collaborate	Download from git-scm.com	<code>git --version</code>

Recommended VS Code Extensions

- Prettier — Code Formatter: Auto-formats your code on save so it always looks consistent
- ESLint: Highlights code errors and style issues before you even run the code
- Prisma: Syntax highlighting and auto-complete for .prisma schema files
- Thunder Client: Test API endpoints from inside VS Code without switching to Postman
- GitLens: Shows who wrote every line of code and when — invaluable for learning
- Error Lens: Shows error messages inline next to the problematic line

2.2 Creating Your First NestJS Project

```
bash — Phase 1

# Install NestJS CLI globally (only do this once per machine)
npm install -g @nestjs/cli

# Create a new NestJS project (this creates a fully structured project)
nest new thunder-erp

# When asked "Which package manager would you like to use?" choose: npm

# Navigate into the project
cd thunder-erp

# Start the development server
# This auto-restarts whenever you save a file — very convenient
npm run start:dev

# Open your browser at:
# http://localhost:3000
# You should see: "Hello World!"

# If you see Hello World, congratulations — your server is running!
```

2.3 Setting Up the Database — Docker + PostgreSQL + Prisma

```
bash — Phase 2
```

```
# _____
```

```
# STEP 1: Start PostgreSQL using Docker
# This downloads PostgreSQL and runs it as a background container
# -----
docker run --name thunder-postgres \
-e POSTGRES_USER=thunder \
-e POSTGRES_PASSWORD=thunder123 \
-e POSTGRES_DB=thundererp \
-p 5432:5432 \
-d postgres:15

# Verify it is running:
docker ps
# You should see a container named "thunder-postgres" with status "Up"

# -----
# STEP 2: Install Prisma in your NestJS project
# -----
npm install prisma @prisma/client
npm install -D prisma

# -----
# STEP 3: Initialize Prisma (creates the prisma/ folder)
# -----
npx prisma init

# This creates:
#   prisma/schema.prisma  - your database schema file
#   .env                  - environment variables file

# -----
# STEP 4: Update your .env file with the database connection
# -----
# Edit .env and set:
# DATABASE_URL="postgresql://thunder:thunder123@localhost:5432/thundererp"

# -----
# STEP 5: Add your first model to prisma/schema.prisma
# (See Section 5 for complete schema examples)
# Then create and apply the migration:
# -----
npx prisma migrate dev --name init

# -----
# STEP 6: Generate the Prisma Client
# Run this every time you change the schema
# -----
```

```
npx prisma generate

# _____
# STEP 7: Open Prisma Studio – a GUI to browse your database
# _____

npx prisma studio
# Opens at http://localhost:5555
# Use this to inspect data, verify your migrations worked, etc.
```

2.4 The .env File — Secrets Management

Sensitive configuration values (database passwords, API keys, JWT secrets) live in the .env file. This file is in .gitignore and must NEVER be committed to Git.

```
.env
# .env – LOCAL ONLY, never commit this file

# PostgreSQL connection string
DATABASE_URL="postgresql://thunder:thunder123@localhost:5432/thundererp"

# JWT secret – used to sign and verify authentication tokens
# In production, use a long random string (64+ characters)
JWT_SECRET="your-super-secret-key-change-this-in-production-use-random-string"

# JWT expiry – how long a token stays valid
JWT_EXPIRES_IN="7d"

# Application port
PORT=3000

# Environment (development | staging | production)
NODE_ENV=development
```

```
.env.example
# .env.example – COMMIT THIS FILE (with fake/empty values)
# This tells new developers what variables they need

DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/DATABASE"
JWT_SECRET="replace-with-a-real-secret"
JWT_EXPIRES_IN="7d"
PORT=3000
NODE_ENV=development
```

SECURITY: Never Commit Secrets

NEVER commit your .env file to Git. It exposes database passwords to everyone.

NEVER paste JWT_SECRET in Slack, email, or any chat system.

NEVER use weak secrets like "password123" or "secret" in any environment.

Always add .env to .gitignore. Always commit .env.example with placeholder values.

3. NestJS Core Architecture — The Building Blocks

NestJS is not just Express with TypeScript. It is an opinionated, enterprise-grade framework that enforces architectural patterns used in large-scale software. It was directly inspired by Angular and uses the same concepts: Modules, Dependency Injection, and Decorators.

3.1 Decorators — The Syntax Behind Everything

A Decorator is a TypeScript feature starting with the @ symbol. Decorators add metadata to classes, methods, and parameters. NestJS reads this metadata to wire everything together automatically.

TypeScript — Decorators

```
// Decorators tell NestJS what each class and method does:
```

```
@Controller("products")    // This class handles routes starting with /products
```

```
@UseGuards(JwtAuthGuard)   // All routes in this class require authentication
```

```
export class ProductController {
```

```
    // Dependency Injection — NestJS provides the service automatically  
    constructor(private readonly productService: ProductService) {}
```

```
    @Get()                  // Handles: GET /products
```

```
    @ApiOperation({ summary: "Get all active products" })
```

```
    findAll() {
```

```
        return this.productService.findAll();
```

```
    }
```

```
    @Get(":id")             // Handles: GET /products/42
```

```
    findOne(
```

```
        @Param("id", ParseIntPipe) id: number, // Converts "42" string to number 42  
    ) {
```

```
        return this.productService.findOne(id);
```

```
    }
```

```
    @Post()                 // Handles: POST /products
```



```
@Roles("admin", "manager")
create(
  @Body() dto: CreateProductDto, // Validates request body against DTO
  @Req() req: Request,           // Full request object (for JWT user)
) {
  return this.productService.create(dto);
}
```

3.2 Modules — Feature Containers (The Departments)

A Module groups related functionality. Every business feature has its own module. Think of modules as departments in a company — Inventory Dept, Sales Dept, Finance Dept. Each is responsible for its own area and has defined interfaces for sharing with others.

TypeScript — Module

```
// src/modules/inventory/inventory.module.ts

import { Module } from "@nestjs/common";
import { InventoryController } from "../inventory.controller";
import { InventoryService } from "../inventory.service";
import { PrismaModule } from "../../prisma/prisma.module";

@Module({
  // imports: Other modules this module needs
  // If you need PrismaService, import PrismaModule here
  imports: [PrismaModule],

  // controllers: All HTTP route handlers for this module
  controllers: [InventoryController],

  // providers: Services, repositories, factories (the business logic)
  providers: [InventoryService],

  // exports: Services that OTHER modules can use
  // If SalesModule needs to check stock, export InventoryService here
  exports: [InventoryService],
})
export class InventoryModule {}

// _____
// CRITICAL RULE:
// If SalesModule needs InventoryService:
// 1. InventoryModule must EXPORT InventoryService
```

```
// 2. SalesModule must IMPORT InventoryModule
// Without both steps, NestJS throws a dependency error
// _____
```

3.3 Controllers — The Gatekeepers (Thin Layer)

Controllers handle HTTP routing ONLY. They receive a request, call a service method, and return the result. Controllers must be THIN — absolutely no business logic. If you find yourself writing "if" statements inside a controller, stop and move that code to the service.

TypeScript — Controller

```
// src/modules/inventory/inventory.controller.ts

import {
  Controller, Get, Post, Put, Delete, Patch,
  Param, Body, Query, Req,
  ParseIntPipe, UseGuards, HttpStatusCode, HttpStatus,
} from "@nestjs/common";
import { InventoryService } from "../inventory.service";
import { CreateProductDto } from "../dto/create-product.dto";
import { UpdateProductDto } from "../dto/update-product.dto";
import { JwtAuthGuard } from "../../auth/guards/jwt-auth.guard";
import { Roles } from "../../auth/decorators/roles.decorator";

@Controller("products") // All routes start with /products
@UseGuards(JwtAuthGuard) // All routes require valid JWT
export class InventoryController {

  constructor(private readonly inventoryService: InventoryService) {}

  // GET /products?page=1&limit=20&search=widget
  @Get()
  findAll(
    @Query("page", ParseIntPipe) page: number = 1,
    @Query("limit", ParseIntPipe) limit: number = 20,
    @Query("search") search?: string,
  ) {
    return this.inventoryService.findAll({ page, limit, search });
  }

  // GET /products/42
  @Get(":id")
  findOne(@Param("id", ParseIntPipe) id: number) {
    return this.inventoryService.findOne(id);
  }
}
```

```
// POST /products - only admin and manager can create
@Post()
@Roles("admin", "manager")
@HttpCode(HttpStatus.CREATED)
create(@Body() dto: CreateProductDto, @Req() req: any) {
  return this.inventoryService.create(dto, req.user.sub);
}

// PUT /products/42 - full update
@Put(":id")
@Roles("admin", "manager")
update(
  @Param("id", ParseIntPipe) id: number,
  @Body() dto: UpdateProductDto,
  @Req() req: any,
) {
  return this.inventoryService.update(id, dto, req.user.sub);
}

// DELETE /products/42 - soft delete, only admin
@Delete(":id")
@Roles("admin")
remove(@Param("id", ParseIntPipe) id: number, @Req() req: any) {
  return this.inventoryService.softDelete(id, req.user.sub);
}
}
```

3.4 Services — The Brain (All Business Logic Lives Here)

Services contain 100% of the business logic. Every calculation, every validation beyond DTO checks, every decision, every database operation — it all goes here. This is where you will spend most of your development time.

TypeScript — Service

```
// src/modules/inventory/inventory.service.ts

import {
  Injectable, NotFoundException,
  BadRequestException, ConflictException,
} from "@nestjs/common";
import { PrismaService } from "../../prisma/prisma.service";
import { CreateProductDto } from "../../dto/create-product.dto";
import { UpdateProductDto } from "../../dto/update-product.dto";
```

```
@Injectable() // Makes this class available for Dependency Injection
export class InventoryService {

  constructor(private readonly prisma: PrismaService) {}

  async findAll(params: { page: number; limit: number; search?: string }) {
    const { page, limit, search } = params;
    const skip = (page - 1) * limit;

    const where = {
      isActive: true,
      ...(search ? {
        OR: [
          { name: { contains: search, mode: "insensitive" as const } },
          { sku: { contains: search, mode: "insensitive" as const } },
        ],
      } : {}),
    };

    const [data, total] = await this.prisma.$transaction([
      this.prisma.product.findMany({ where, skip, take: limit, orderBy: { name: "asc" } }),
      this.prisma.product.count({ where }),
    ]);

    return { data, total, page, limit, totalPages: Math.ceil(total / limit) };
  }

  async findOne(id: number) {
    const product = await this.prisma.product.findUnique({
      where: { id, isActive: true },
    });
    if (!product) {
      throw new NotFoundException(`Product #${id} not found or has been deleted`);
    }
    return product;
  }

  async create(dto: CreateProductDto, createdById: number) {
    // Business rule: SKU must be globally unique
    const existing = await this.prisma.product.findUnique({ where: { sku: dto.sku } });
    if (existing) {
      throw new ConflictException(`A product with SKU "${dto.sku}" already exists`);
    }
    return this.prisma.product.create({ data: { ...dto, createdById } });
  }
}
```

```
}

async update(id: number, dto: UpdateProductDto, updatedById: number) {
  await this.findOne(id); // Throws 404 if not found
  return this.prisma.product.update({ where: { id }, data: dto });
}

async softDelete(id: number, deletedById: number) {
  await this.findOne(id); // Verify it exists
  return this.prisma.product.update({
    where: { id },
    data: { isActive: false },
  });
}
}
```

3.5 Dependency Injection — Let NestJS Wire Your Code

Dependency Injection (DI) means you declare what your class needs, and NestJS provides it automatically. You never manually create instances of services.

TypeScript — Dependency Injection

```
// WRONG — Never do this:
export class InventoryController {
  // You manually create the service — tightly coupled, untestable
  private service = new InventoryService(new PrismaService());
}

// CORRECT — Dependency Injection:
export class InventoryController {
  // Declare what you need. NestJS provides the singleton instance.
  constructor(private readonly inventoryService: InventoryService) {}
}

// Why DI matters:
// 1. Singleton: NestJS creates ONE instance shared by all that use it
// 2. Testable: In tests, inject a MOCK service to test in isolation
// 3. Loosely coupled: Controller does not know or care how Service works
// 4. Automatic lifecycle: NestJS handles creation, initialisation, cleanup
```

4. DTOs & Input Validation — Never Trust the Frontend

A DTO (Data Transfer Object) is a TypeScript class that defines the exact shape and constraints of data coming into your API. It is your first line of defence. Every single API endpoint that receives data must have a DTO.

Why DTOs Are Non-Negotiable — A Real Attack

Without DTO validation, a malicious user can POST this body to /products:

```
{ "name": "Free Widget", "price": -999.99, "quantity": 999999, "isAdmin": true }
```

Without validation: your system creates a product with a negative price.

With class-validator DTOs: the request is automatically rejected with a 400 error.

The frontend cannot protect you. Anyone with Postman or curl can bypass it entirely.

4.1 Setting Up Global Validation

```
bash
```

```
# Install validation libraries
```

```
npm install class-validator class-transformer
```

```
TypeScript — main.ts
```

```
// src/main.ts — Enable global validation (do this ONCE)
```

```
import { ValidationPipe } from "@nestjs/common";
```

```
import { NestFactory } from "@nestjs/core";
```

```
import { AppModule } from "../app.module";
```

```
async function bootstrap() {
```

```
  const app = await NestFactory.create(AppModule);
```

```
  app.useGlobalPipes(new ValidationPipe({
```

```
    whitelist: true, // Strip properties not in DTO (security)
```

```
    forbidNonWhitelisted: true, // Reject requests with unknown properties
```

```
    transform: true, // Auto-convert types (string "42" becomes number 42)
```

```
    transformOptions: {
```

```
      enableImplicitConversion: true, // Even without @Type() decorator
```

```
    },
```

```
  }));
```

```
  await app.listen(process.env.PORT ?? 3000);
```

```
  console.log(`Application running on: ${await app.getUrl()}`);
```

```
}
```

```
bootstrap();
```

4.2 Writing a Complete DTO with All Validators

TypeScript — DTO

```
// src/modules/inventory/dto/create-product.dto.ts

import {
  IsString, IsNumber, IsPositive, IsOptional,
  IsEnum, MinLength, MaxLength, Min, Max,
  IsInt, IsNotEmpty, IsDecimal, ArrayMinSize,
} from "class-validator";
import { ApiProperty, ApiPropertyOptional } from "@nestjs/swagger";

export enum ProductCategory {
  ELECTRONICS = "ELECTRONICS",
  CLOTHING    = "CLOTHING",
  FOOD        = "FOOD",
  FURNITURE   = "FURNITURE",
  OTHER       = "OTHER",
}

export class CreateProductDto {

  @ApiProperty({ example: "Widget Pro Max", description: "Display name of the product" })
  @IsString()           // Must be a string
  @IsNotEmpty()         // Cannot be empty string ""
  @MinLength(2, { message: "Name must be at least 2 characters long" })
  @MaxLength(100, { message: "Name cannot exceed 100 characters" })
  name: string;

  @ApiProperty({ example: "WGT-PRO-001", description: "Unique Stock Keeping Unit" })
  @IsString()
  @IsNotEmpty()
  @MinLength(3)
  @MaxLength(50)
  sku: string;

  @ApiProperty({ example: 29.99, description: "Selling price in INR. Must be positive." })
  @IsNumber({ maxDecimalPlaces: 2 }, { message: "Price can have at most 2 decimal places" })
  @IsPositive({ message: "Price must be greater than zero" })
  price: number;
```

```
@ApiProperty({ example: 100, description: "Initial stock quantity. Cannot be negative." })
@IsInt({ message: "Quantity must be a whole number" })
@Min(0, { message: "Quantity cannot be negative" })
@Max(999999)
quantity: number;

@ApiProperty({ enum: ProductCategory, example: "ELECTRONICS" })
@IsEnum(ProductCategory, { message: "Category must be one of: ELECTRONICS, CLOTHING, FOOD, FURNITURE, OTHER" })
category: ProductCategory;

@ApiPropertyOptional({ example: "High-performance widget with pro features" })
@IsOptional() // This field is optional – can be omitted entirely
@IsString()
@MaxLength(500)
description?: string;
}
```

4.3 Update DTOs with PartialType

When updating a record, all fields should be optional. Use `PartialType` so you do not duplicate your validation rules:

TypeScript — Update DTO

```
// src/modules/inventory/dto/update-product.dto.ts

import { PartialType } from "@nestjs/mapped-types";
import { CreateProductDto } from "../create-product.dto";

// ALL fields from CreateProductDto become optional.
// But if a field IS provided, all validators still apply.
// So if you send { price: -5 }, it still fails with 400.
export class UpdateProductDto extends PartialType(CreateProductDto) {}

// Now PATCH /products/42 with body { "price": 34.99 } is valid.
// And PATCH /products/42 with body { "price": -1 } returns 400.
```

5. Prisma ORM — Type-Safe Database Interaction

Prisma is the bridge between TypeScript and PostgreSQL. It generates TypeScript types from your database schema, meaning TypeScript knows exactly what shape every database query returns. This catches database errors at compile time, not at runtime.

5.1 The Prisma Schema — Source of Truth

The `prisma/schema.prisma` file is the single source of truth for your database structure. Every table, column, relationship, and constraint is defined here. Never create tables by hand in SQL — define them in the schema and let Prisma generate the migration.

```
Prisma Schema
// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL") // Reads from .env
}

// — USER TABLE —————
model User {
  id          Int          @id @default(autoincrement())
  name        String
  email       String       @unique // No two users can have same email
  passwordHash String
  role        UserRole     @default(STAFF)
  isActive    Boolean      @default(true)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt // Auto-updates on every change

  // Relationships - things this user has created
  products    Product[]
  orders       Order[]
  auditLogs   AuditLog[]

  @@map("users") // Actual PostgreSQL table name
}

// — PRODUCT TABLE —————
model Product {
  id          Int          @id @default(autoincrement())
  sku         String       @unique
  name        String
```

```

description String?          // ? means nullable (optional)
price          Decimal        @db.Decimal(10, 2)  // Up to 9,999,999.99
quantity       Int            @default(0)
category       ProductCategory @default(OTHER)
isActive       Boolean        @default(true)
createdAt      DateTime        @default(now())
updatedAt      DateTime        @updatedAt

// Foreign key to User
createdBy      User            @relation(fields: [createdById], references: [id])
createdById    Int

// Relationship to OrderItems
orderItems     OrderItem[]

@@map("products")
}

// — ORDER TABLE —————
model Order {
  id          Int          @id @default(autoincrement())
  orderNumber String        @unique
  status       OrderStatus @default(PENDING)
  totalAmount Decimal       @db.Decimal(10, 2)
  notes        String?
  isActive     Boolean      @default(true)
  createdAt    DateTime     @default(now())
  updatedAt    DateTime     @updatedAt

  customer     Customer      @relation(fields: [customerId], references: [id])
  customerId    Int
  createdBy     User          @relation(fields: [createdById], references: [id])
  createdById  Int

  // One order has many order items
  items         OrderItem[]

  @@map("orders")
}

// — ORDER ITEMS (many-to-many join) —————
model OrderItem {
  id          Int          @id @default(autoincrement())
  quantity     Int
  unitPrice    Decimal @db.Decimal(10, 2)  // Price AT TIME OF ORDER (not current price)

```

```

order      Order    @relation(fields: [orderId], references: [id])
orderId    Int
product    Product  @relation(fields: [productId], references: [id])
productId  Int

@@unique([orderId, productId]) // Cannot add same product twice to an order
@@map("order_items")
}

// — AUDIT LOG —————
model AuditLog {
  id          Int      @id @default(autoincrement())
  entityType  String    // "Product", "Order", "User"
  entityId    Int
  action      String    // "CREATE", "UPDATE", "DELETE", "STATUS_CHANGE"
  oldValue    String?   // JSON of previous state
  newValue    String?   // JSON of new state
  performedBy Int
  user        User      @relation(fields: [performedBy], references: [id])
  createdAt   DateTime @default(now())
  @@map("audit_logs")
}

// — ENUMS —————
enum OrderStatus { PENDING CONFIRMED SHIPPED DELIVERED COMPLETED CANCELLED }
enum ProductCategory { ELECTRONICS CLOTHING FOOD FURNITURE OTHER }
enum UserRole { ADMIN MANAGER STAFF }

```

5.2 The Shared PrismaService

```

TypeScript — PrismaService
// src/modules/prisma/prisma.service.ts
// Create this ONCE and import PrismaModule everywhere else

import { Injectable, OnModuleInit, OnModuleDestroy } from "@nestjs/common";
import { PrismaClient } from "@prisma/client";

@Injectable()
export class PrismaService extends PrismaClient
  implements OnModuleInit, OnModuleDestroy {

  async onModuleInit() {
    // Connects to PostgreSQL when the NestJS app starts

```

```
    await this.$connect();
  }

  async onModuleDestroy() {
    // Disconnects cleanly when app shuts down (prevents connection leaks)
    await this.$disconnect();
  }
}

// src/modules/prisma/prisma.module.ts
import { Global, Module } from "@nestjs/common";
import { PrismaService } from "../prisma.service";

@Global() // Makes PrismaModule available everywhere without importing it in every module
@Module({ providers: [PrismaService], exports: [PrismaService] })
export class PrismaModule {}
```

5.3 Complete Prisma Query Reference

TypeScript — Prisma Queries

```
// — READING DATA —————

// Find all (with filters, sorting, pagination)
const products = await this.prisma.product.findMany({
  where: { isActive: true, category: "ELECTRONICS" },
  orderBy: [{ name: "asc" }],
  take: 20, // Limit: max 20 results
  skip: 40, // Offset: skip first 40 (page 3)
  select: { id: true, name: true, price: true }, // Only these fields
});

// Find one by unique field (returns null if not found)
const product = await this.prisma.product.findUnique({
  where: { id: 42 },
  include: { orderItems: true, createdBy: true }, // Load relations
});

// Find one and throw if not found (preferred in services)
const product = await this.prisma.product.findUniqueOrThrow({
  where: { id: 42 },
});

// Search with text contains (case-insensitive)
const results = await this.prisma.product.findMany({
```

```
where: {
  OR: [
    { name: { contains: "widget", mode: "insensitive" } },
    { sku: { contains: "widget", mode: "insensitive" } },
  ],
  price: { gte: 10.00, lte: 500.00 }, // Between $10 and $500
  quantity: { gt: 0 }, // In stock
},
});

// Count records
const total = await this.prisma.product.count({
  where: { isActive: true },
});

// — CREATING DATA —————

const newProduct = await this.prisma.product.create({
  data: {
    name: "Widget Pro",
    sku: "WGT-001",
    price: 29.99,
    quantity: 100,
    category: "ELECTRONICS",
    createdById: userId,
  },
});

// — UPDATING DATA —————

// Standard update
const updated = await this.prisma.product.update({
  where: { id: 42 },
  data: { price: 34.99 },
});

// Atomic increment/decrement (SAFE for concurrent access)
await this.prisma.product.update({
  where: { id: 42 },
  data: { quantity: { decrement: 5 } }, // quantity = quantity - 5
});

// Update many at once
await this.prisma.product.updateMany({
  where: { category: "FOOD" },
  data: { price: { multiply: 1.05 } }, // 5% price increase on all food
```

```
});  
  
// — SOFT DELETE —————  
  
// NEVER use this.prisma.product.delete() in production  
// ALWAYS use soft delete:  
await this.prisma.product.update({  
  where: { id: 42 },  
  data: { isActive: false },  
});
```

5.4 Migrations — Tracking Database Changes

```
bash — Migrations  
# —————  
# Development workflow  
# —————  
  
# After editing schema.prisma, create and apply a migration:  
npx prisma migrate dev --name add_product_discount_column  
  
# This command:  
# 1. Generates SQL for the schema changes  
# 2. Applies the SQL to your local database  
# 3. Creates a migration file in prisma/migrations/  
# 4. Runs prisma generate automatically  
  
# —————  
# Production deployment  
# —————  
  
# Apply pending migrations WITHOUT creating new ones:  
npx prisma migrate deploy  
  
# —————  
# Useful commands  
# —————  
  
npx prisma migrate status      # See which migrations are applied  
npx prisma studio             # Visual database browser at localhost:5555  
npx prisma generate           # Regenerate TypeScript client after schema changes  
  
# DANGER: Only use in development. Drops ALL data.  
npx prisma migrate reset
```

Migration Rules

Never manually edit generated migration files in prisma/migrations/.
Always run "npx prisma generate" after any schema change.
Commit ALL migration files to Git — they are part of your codebase.
Never run "migrate reset" on production. It deletes everything.
Always run "migrate deploy" (not migrate dev) on production servers.

6. Error Handling & HTTP Status Codes

Every API must return meaningful error responses. NestJS maps thrown exceptions to HTTP responses automatically. Use the correct HTTP status code — it communicates what went wrong to both the frontend developer and the end user.

6.1 The HTTP Status Code Reference

NestJS Exception	HTTP Code	When To Use
BadRequestException	400 Bad Request	Input passes DTO format but fails business rules (e.g., quantity negative)
UnauthorizedException	401 Unauthorized	No JWT token provided, or token is invalid or expired
ForbiddenException	403 Forbidden	User is authenticated but does not have the required role/permission
NotFoundException	404 Not Found	The requested resource with that ID does not exist or is soft-deleted
ConflictException	409 Conflict	Duplicate data (e.g., SKU already exists, email already registered)
UnprocessableEntityException	422 Unprocessable	Format is valid, but semantic rules are violated (complex business logic)
InternalServerErrorException	500 Server Error	Unexpected error not caught elsewhere (use very sparingly)

6.2 Using Exceptions in Services

```
TypeScript — Error Handling
```

```
// src/modules/inventory/inventory.service.ts
```

```
import {
  NotFoundException,
  ConflictException,
  BadRequestException,
} from "@nestjs/common";

async findOne(id: number) {
  const product = await this.prisma.product.findUnique({
    where: { id, isActive: true },
  });

  if (!product) {
    // NestJS automatically converts this to HTTP 404:
    // { "statusCode": 404, "message": "Product #42 not found", "error": "Not Found" }
    throw new NotFoundException(`Product #${id} not found`);
  }

  return product;
}

async reduceStock(productId: number, quantity: number) {
  const product = await this.findOne(productId); // Throws 404 automatically

  if (quantity <= 0) {
    throw new BadRequestException("Quantity to reduce must be greater than zero");
  }

  if (product.quantity < quantity) {
    throw new BadRequestException(
      `Insufficient stock. Available: ${product.quantity}, Requested: ${quantity}`
    );
  }

  return this.prisma.product.update({
    where: { id: productId },
    data: { quantity: { decrement: quantity } },
  });
}
```

7. Authentication & Security (JWT + RBAC)

Authentication answers "Who are you?" Authorization answers "What are you allowed to do?" Both are critical in any ERP system where different roles have different access levels.

7.1 How JWT Authentication Works



7.2 Installing and Setting Up Auth

```
bash
```

```
# Install all required packages
npm install @nestjs/passport @nestjs/jwt passport passport-jwt bcrypt
npm install -D @types/passport-jwt @types/bcrypt
```

```
TypeScript - AuthService
```

```
// src/modules/auth/auth.service.ts

import { Injectable, UnauthorizedException } from "@nestjs/common";
import { JwtService } from "@nestjs/jwt";
import { PrismaService } from "../../prisma/prisma.service";
import * as bcrypt from "bcrypt";

const SALT_ROUNDS = 12; // Higher = more secure but slower. 12 is the production
standard.

@Injectable()
export class AuthService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly jwtService: JwtService,
  ) {}

  async login(email: string, password: string) {
    const user = await this.prisma.user.findUnique({
      where: { email, isActive: true },
    });

    // SECURITY: Use identical error for "user not found" AND "wrong password"
    // Never reveal WHICH one failed — that gives attackers information
    const isValid = user && await bcrypt.compare(password, user.passwordHash);
    if (!isValid) {
      throw new UnauthorizedException("Invalid email or password");
    }

    // JWT payload — this data is encoded in the token (readable, not secret)
    const payload = { sub: user.id, email: user.email, role: user.role };

    return {
      access_token: this.jwtService.sign(payload),
      user: { id: user.id, name: user.name, email: user.email, role: user.role },
    };
  }
}
```

```
async register(name: string, email: string, password: string) {
  const passwordHash = await bcrypt.hash(password, SALT_ROUNDS);
  return this.prisma.user.create({
    data: { name, email, passwordHash },
    select: { id: true, name: true, email: true }, // NEVER return passwordHash
  });
}
```

7.3 Role-Based Access Control (RBAC)

TypeScript — RBAC

```
// src/modules/auth/decorators/roles.decorator.ts

import { SetMetadata } from "@nestjs/common";
export const ROLES_KEY = "roles";
export const Roles = (...roles: string[]) => SetMetadata(ROLES_KEY, roles);

// _____
// Usage in any controller:
// _____

@Controller("products")
@UseGuards(JwtAuthGuard, RolesGuard)
export class ProductController {

  @Get() // No @Roles = any authenticated user
  findAll() { ... }

  @Post()
  @Roles("admin", "manager") // Only these roles can create
  create(...) { ... }

  @Delete(":id")
  @Roles("admin") // Only admin can delete
  remove(...) { ... }
}

// The RolesGuard reads the @Roles metadata and checks req.user.role
```

8. Database Transactions — Keeping Data Consistent

A transaction is a group of database operations that either ALL succeed together, or ALL fail and roll back together. In ERP systems, almost every meaningful action involves multiple database writes. Without transactions, a failure halfway through leaves your data in a corrupt, inconsistent state.

Why Transactions Are Critical — Real Scenario

Sales rep creates an order. The system must:

1. Create the Order record
2. Create OrderItem records for each product
3. Deduct stock from each product
4. Create an Invoice record

WITHOUT transaction: If step 3 fails (e.g., database timeout), the order exists in the database but no stock was deducted. The ERP now shows wrong inventory.

WITH transaction: If step 3 fails, ALL previous steps are rolled back. The database returns to its exact state before the operation started.

8.1 Implementing Transactions in Prisma

TypeScript — Transactions

```
// src/modules/orders/orders.service.ts

async createOrder(dto: CreateOrderDto, createdById: number) {

  // PHASE 1: Validate everything BEFORE starting the transaction
  // (Faster to fail early than to start a transaction and then fail)
  const customer = await this.prisma.customer.findUniqueOrThrow({
    where: { id: dto.customerId },
  });

  let totalAmount = 0;
  const itemsWithPrices = [];

  for (const item of dto.items) {
    const product = await this.prisma.product.findUniqueOrThrow({
      where: { id: item.productId, isActive: true },
    });

    if (product.quantity < item.quantity) {
      throw new BadRequestException(
        `Insufficient stock for "${product.name}". ` +
        `Available: ${product.quantity}, Requested: ${item.quantity}`
      );
    }
  }
}
```

```
    );
  }

  const lineTotal = Number(product.price) * item.quantity;
  totalAmount += lineTotal;
  itemsWithPrices.push({ ...item, unitPrice: product.price });
}

// PHASE 2: Execute all writes in a single atomic transaction
return this.prisma.$transaction(async (tx) => {
  // Note: "tx" is a Prisma client scoped to this transaction
  // Use "tx" for ALL database operations inside here, not "this.prisma"

  // Step 1: Create the order
  const order = await tx.order.create({
    data: {
      orderNumber: `ORD-${Date.now()}`,
      totalAmount,
      customerId: dto.customerId,
      createdById,
    },
  });

  // Step 2: Create all order items
  for (const item of itemsWithPrices) {
    await tx.orderItem.create({
      data: {
        orderId: order.id,
        productId: item.productId,
        quantity: item.quantity,
        unitPrice: item.unitPrice, // Store price at time of order!
      },
    });
  }

  // Step 3: Deduct stock atomically
  await tx.product.update({
    where: { id: item.productId },
    data: { quantity: { decrement: item.quantity } },
  });
}

// Step 4: Create audit log entry
await tx.auditLog.create({
  data: {
    entityType: "Order",
    entityId: order.id,
```

```

    action:      "CREATE",
    newValue:    JSON.stringify({ totalAmount, itemCount: dto.items.length }),
    performedBy: createdById,
  },
});

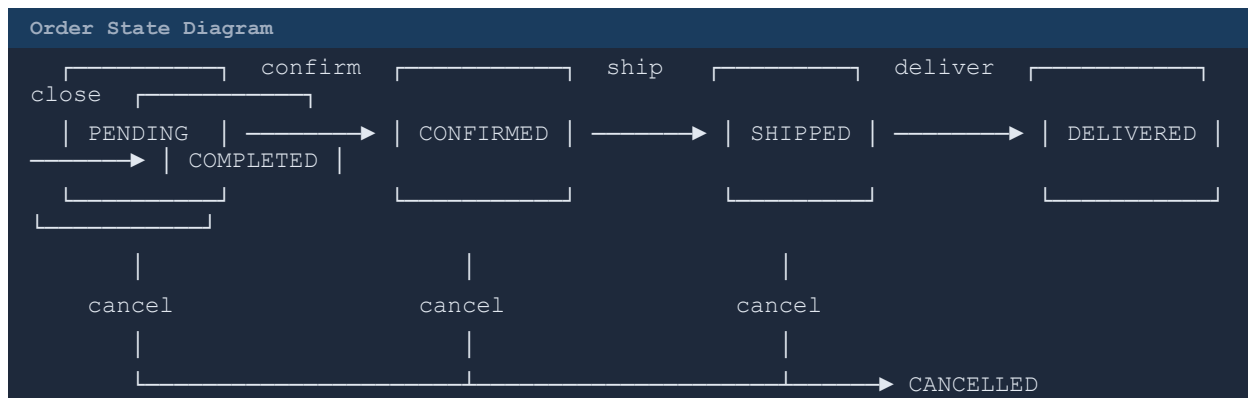
// Return the complete order with all relations loaded
return tx.order.findUniqueOrThrow({
  where: { id: order.id },
  include: {
    items:      { include: { product: true } },
    customer: true,
  },
});

// If ANY operation above throws, ALL are automatically rolled back
});
}

```

9. State Machines — Managing Business Object Lifecycle

In an ERP, every business object has a lifecycle. An order cannot jump from PENDING to DELIVERED — it must pass through every intermediate state. A state machine enforces these rules in code.



TypeScript — State Machine

```

// src/modules/orders/orders.service.ts

import { OrderStatus } from "@prisma/client";

// Define which transitions are allowed from each state

```

```
const VALID_TRANSITIONS: Record<OrderStatus, OrderStatus[]> = {
  PENDING: [OrderStatus.CONFIRMED, OrderStatus.CANCELLED],
  CONFIRMED: [OrderStatus.SHIPPED, OrderStatus.CANCELLED],
  SHIPPED: [OrderStatus.DELIVERED, OrderStatus.CANCELLED],
  DELIVERED: [OrderStatus.COMPLETED],
  COMPLETED: [], // Terminal state - no further transitions
  CANCELLED: [], // Terminal state
};

async transitionStatus(orderId: number, newStatus: OrderStatus, userId: number) {
  const order = await this.prisma.order.findUniqueOrThrow({
    where: { id: orderId },
  });

  const allowed = VALID_TRANSITIONS[order.status] ?? [];

  if (!allowed.includes(newStatus)) {
    throw new BadRequestException(
      `Cannot transition order from ${order.status} to ${newStatus}. ` +
      `Allowed transitions: [${allowed.join(", ")} || "none - this is a terminal state"]`
    );
  }

  return this.prisma.$transaction(async (tx) => {
    const updated = await tx.order.update({
      where: { id: orderId },
      data: { status: newStatus },
    });

    // Always log state changes for audit trail
    await tx.auditLog.create({
      data: {
        entityType: "Order",
        entityId: orderId,
        action: "STATUS_CHANGE",
        oldValue: order.status,
        newValue: newStatus,
        performedBy: userId,
      },
    });

    return updated;
  });
}
```

10. Concurrency & Race Conditions

Concurrency bugs are the most dangerous type because they are intermittent — they only appear under load, when two users act simultaneously. They are impossible to reproduce reliably in development.

Race Condition — Without Locking

Time →

Thread A (User 1 buying last item):

```
READ  stock = 1 ✓
CHECK 1 >= 1 → allow ✓
WRITE stock = 0 ✓
```

Thread B (User 2 buying last item at same moment):

```
READ  stock = 1 ✓ (reads BEFORE Thread A writes)
CHECK 1 >= 1 → allow ✓
WRITE stock = -1 ✗✗✗ CRITICAL BUG — Stock is now negative!
```

10.1 The Safe Solution — Conditional Atomic Updates

TypeScript — Concurrency Safe

```
// SAFE: Use a conditional WHERE clause in the UPDATE itself
// The database processes this as a single atomic operation
// No window for a race condition between the read and the write

async reduceStockSafely(productId: number, qty: number): Promise<void> {

  // This SQL executes atomically:
  // UPDATE products SET quantity = quantity - {qty}
  // WHERE id = {productId} AND quantity >= {qty}
  const affectedRows = await this.prisma.$executeRaw`
    UPDATE products
    SET quantity = quantity - ${qty}
    WHERE id = ${productId}
      AND quantity >= ${qty}
      AND is_active = true
  `;

  if (affectedRows === 0) {
    // Two possible reasons:
    // 1. Product not found
    // 2. Insufficient stock (another thread bought it first)
```



```
    throw new BadRequestException(  
      "Insufficient stock — item may have been purchased by another user"  
    );  
  }  
}
```

11. API Documentation with Swagger

Swagger generates an interactive API documentation page automatically. Every endpoint you build must be documented with Swagger so the frontend team knows how to call it.

```
bash  
npm install @nestjs/swagger swagger-ui-express
```

TypeScript — Swagger Setup

```
// src/main.ts — Add Swagger (AFTER ValidationPipe setup)  
  
import { SwaggerModule, DocumentBuilder } from "@nestjs/swagger";  
  
const swaggerConfig = new DocumentBuilder()  
  .setTitle("ThunderERP API")  
  .setDescription("Complete REST API documentation for ThunderERP. All endpoints  
require a Bearer JWT token unless marked public.")  
  .setVersion("1.0")  
  .addBearerAuth() // Adds "Authorize" button so you can test protected routes  
  .addTag("products", "Product and inventory management")  
  .addTag("orders", "Order lifecycle management")  
  .addTag("customers", "Customer and CRM data")  
  .addTag("auth", "Authentication — login, register")  
  .build();  
  
const document = SwaggerModule.createDocument(app, swaggerConfig);  
SwaggerModule.setup("api", app, document);  
  
// Now visit: http://localhost:3000/api  
// You will see all your endpoints with their request/response schemas
```

12. Folder Structure — Industry Standard

Project Folder Structure

```
thunder-erp/
|
|└─ prisma/
|  |└─ schema.prisma          # THE source of truth for your database
|  └─ migrations/            # Auto-generated. Never edit manually. Always
commit.
|
|└─ src/
|  |└─ main.ts                # Entry point: sets up ValidationPipe, Swagger,
starts server
|  |└─ app.module.ts          # Root module: imports all feature modules
|  |
|  |└─ common/                # Shared code used across multiple modules
|  |  |└─ decorators/
|  |  |  └─ current-user.decorator.ts # @CurrentUser() shortcut
|  |  └─ filters/
|  |    └─ http-exception.filter.ts   # Global error formatting
|  |  └─ guards/
|  |    |└─ jwt-auth.guard.ts         # Validates JWT token
|  |    └─ roles.guard.ts             # Checks role permissions
|  |  └─ interceptors/
|  |    └─ logging.interceptor.ts     # Logs every request and response
|  |
|  └─ modules/
|    |
|    |└─ prisma/              # Database connection (import this everywhere)
|    |  |└─ prisma.module.ts
|    |  └─ prisma.service.ts
|    |
|    |└─ auth/
|    |  |└─ auth.module.ts
|    |  |└─ auth.controller.ts      # POST /auth/login, /auth/register
|    |  |└─ auth.service.ts
|    |  └─ dto/
|    |    |└─ login.dto.ts
|    |    └─ register.dto.ts
|    |  └─ guards/
|    |    └─ jwt-auth.guard.ts
|    |  └─ strategies/
|    |    └─ jwt.strategy.ts        # Passport JWT strategy
|    |
|    |└─ inventory/
|    |  |└─ inventory.module.ts
|    |  |└─ inventory.controller.ts  # GET/POST/PUT/DELETE /products
|    |  |└─ inventory.service.ts
|    |  └─ dto/
|    |    └─ create-product.dto.ts
```

```

├── update-product.dto.ts
├── orders/
│   ├── orders.module.ts
│   ├── orders.controller.ts
│   ├── orders.service.ts
│   └── dto/
│       ├── create-order.dto.ts
│       └── update-order-status.dto.ts
├── customers/
├── users/
└── audit/
    ├── audit.module.ts
    └── audit.service.ts

test/                                # End-to-end tests
.env                                # Secrets. NEVER commit.
.env.example                        # Template with fake values. ALWAYS commit.
.gitignore                         # Must include: node_modules, .env, dist
package.json
tsconfig.json

```

13. Common Mistakes & How to Avoid Them

Mistake	Why It Is Wrong	The Correct Approach
Business logic in Controllers	Breaks separation of concerns. Cannot be unit tested independently.	Move ALL logic to Services. Controllers only route.
Using TypeScript "any"	Defeats the entire purpose of TypeScript. Hides bugs at compile time.	Always define proper interfaces, types, or classes.
No input validation	Security vulnerability. Bad data reaches your database.	Use class-validator decorators in every DTO.
No transactions	Partial failures corrupt data across multiple tables.	Use prisma.\$transaction() for any multi-step write operation.
Hard deleting records	Permanent, unrecoverable data loss. Historical records broken.	Soft delete with isActive = false. Never run DELETE.
Not auditing changes	Cannot trace who changed what and when. Legal compliance risk.	Log every mutation to AuditLog inside the same transaction.

Mistake	Why It Is Wrong	The Correct Approach
Missing await on async calls	Returns a Promise object instead of the actual data. Silent bug.	Always await every async operation. TypeScript will warn you.
Plain text passwords	Security catastrophe. Database breach = all users compromised.	Always hash with bcrypt and SALT_ROUNDS of 10 or higher.
Committing .env files	Exposes all secrets (DB password, JWT key) to the entire team.	Add .env to .gitignore. Commit only .env.example.
No pagination	Returns millions of rows. Server runs out of memory and crashes.	Always use take + skip. Always return total count.
Storing price only on Product	When product price changes, old order totals become wrong.	Store unitPrice on OrderItem at time of order creation.

14. Development Workflow — Building a Feature

Follow this process for every new feature. Do not skip steps.

1. Understand the requirement fully. Read the task. Draw the data model on paper. Ask questions before writing code.
2. Update prisma/schema.prisma — add new models or fields needed for the feature.
3. Run migration: `npx prisma migrate dev --name describe-what-changed`
4. Generate CLI scaffold: `nest g res modules/feature-name` (creates module, controller, service, DTOs)
5. Define your DTOs — create `create-*.dto.ts` and `update-*.dto.ts` with all validators and Swagger annotations.
6. Implement the Service — write business logic. Handle happy path first, then all error cases.
7. Update the Controller — add routes, guards, `@Roles` decorators, and Swagger annotations.
8. Test with Postman — test every route. Test happy path AND error cases (invalid data, wrong role, missing ID).
9. Add audit logging — ensure every mutation (create, update, status change, soft delete) is logged.
10. Review your own code — read it as if you are the next developer joining the team.
11. Get a code review from a team member before merging.

14.1 NestJS CLI Reference

```
bash - NestJS CLI
```

```
# Generate a complete CRUD resource (most common command)
# Creates: module, controller, service, dto folder
nest g res modules/inventory

# Generate individual files
nest g module      modules/customers
nest g controller modules/customers --no-spec # Skip test file
nest g service     modules/customers --no-spec

# Generate a guard, decorator, interceptor, or filter
nest g guard       common/guards/jwt-auth
nest g decorator   common/decorators/current-user
nest g interceptor  common/interceptors/logging
nest g filter       common/filters/http-exception
```

15. TypeScript Essentials for NestJS

TypeScript is mandatory in NestJS. These are the most important concepts you need to understand:

15.1 Why TypeScript — The Practical Benefit

TypeScript — Why It Matters

```
// WITHOUT TypeScript (plain JavaScript):
async function findProduct(id) {
  const product = await db.findOne(id);
  return product.nmae; // Typo - no error caught until runtime!
}

// WITH TypeScript:
async function findProduct(id: number): Promise<Product> {
  const product = await this.prisma.product.findUniqueOrThrow({ where: { id } });
  return product.nmae; // TypeScript ERROR at compile time: "nmae" does not exist
  //                  // Property "name" did you mean?
}

// TypeScript catches bugs BEFORE you run the code.
// With Prisma, TypeScript also knows exactly what .findMany() returns.
```

15.2 Async/Await — The Most Common Bug Source

TypeScript — Async/Await

```
// The most common and subtle bug for new NestJS developers:

// WRONG — Missing await:
async createOrder(dto: CreateOrderDto) {
  const customer = this.prisma.customer.findUnique({ where: { id: dto.customerId }
});
  //           ^ No await!
  // "customer" is a PROMISE object, not the actual customer data

  if (!customer) throw new NotFoundException();
  // This will NEVER throw — a Promise object is always truthy
}

// CORRECT:
async createOrder(dto: CreateOrderDto) {
  const customer = await this.prisma.customer.findUnique({ where: { id:
dto.customerId } });
  //           ^ With await, customer is the actual database result

  if (!customer) throw new NotFoundException("Customer not found");
  // Now this correctly throws when customer does not exist
}

// Running multiple queries in PARALLEL (faster than sequential):
const [products, customers, totalOrders] = await Promise.all([
  this.prisma.product.findMany(),
  this.prisma.customer.findMany(),
  this.prisma.order.count(),
]);
// All three queries run simultaneously, not one after the other.
```

15.3 Useful TypeScript Patterns in NestJS

TypeScript — Common Patterns

```
// Interface for JWT payload (data inside the token)
interface JwtPayload {
  sub: number; // User ID (JWT convention: "subject")
  email: string;
  role: string;
  iat: number; // Issued at (timestamp)
  exp: number; // Expiry (timestamp)
}
```

```
// Generic paginated response type
interface PaginatedResult<T> {
  data:      T[];
  total:     number;
  page:      number;
  limit:     number;
  totalPages: number;
}

// Partial — make all fields optional (Prisma uses this a lot)
type UpdateData = Partial<CreateProductDto>;

// Pick — select only specific fields from a type
type ProductSummary = Pick<Product, "id" | "name" | "price">;

// Omit — exclude specific fields from a type
type ProductWithoutTimestamps = Omit<Product, "createdAt" | "updatedAt">;

// Record — dictionary with typed keys and values
const transitions: Record<OrderStatus, OrderStatus[]> = { ... };
```

16. The Golden Rules of ThunderERP

Rule	What It Means	Why It Matters
1. Logic in Services only	No business logic in controllers. Ever.	Testability, separation of concerns, maintainability
2. Never use "any" type	Define proper TypeScript types for everything	Catches bugs at compile time instead of production
3. Always validate input	Every endpoint with a body must have a DTO	Security, data integrity, clear API contracts
4. Use transactions	Multi-step writes always use <code>prisma.\$transaction()</code>	Data consistency, no partial failure corruption
5. Soft delete only	Set <code>isActive = false</code> . Never use <code>prisma.X.delete()</code>	Data preservation, history, regulatory compliance
6. Audit everything	Every mutation creates an <code>AuditLog</code> entry	Traceability, debugging, accountability
7. Handle errors explicitly	Throw the correct NestJS exception with a clear message	Better debugging, better UX for frontend
8. Never commit secrets	<code>.env</code> files never go to Git. Use <code>.env.example</code> instead	Security — prevents credential exposure
9. Always paginate lists	Every <code>findMany()</code> has <code>take + skip</code> limits	Performance — never return unlimited rows

Rule	What It Means	Why It Matters
10. Store prices at point in time	Save price on OrderItem, not just on Product	Historical accuracy — price changes do not corrupt old orders
11. Consistency over cleverness	Follow established patterns. Do not invent your own.	The next developer (or future you) can understand it

17. Learning Roadmap

Week	Topics	Hands-On Goal	You Know It When...
Week 1	NestJS basics, modules, controllers, services, simple DTOs	Build working CRUD API for Products (no database, use in-memory array first)	You can explain Controller vs Service in your own words
Week 2	Prisma ORM, migrations, relationships, pagination	Connect Products API to PostgreSQL. Add Categories model. Add pagination.	You can write a Prisma query with include and where without looking it up
Week 3	JWT auth, guards, RBAC, secure endpoints	Add login/register. Protect all routes. Add role-based access.	You can explain what is in a JWT and how the guard validates it
Week 4	Transactions, state machines, concurrency, audit logging	Build Order creation with full stock deduction and audit trail	You can implement a multi-step transaction from memory
Week 5+	Swagger, error handling, soft deletes, production patterns	Build a complete feature from scratch: requirement to tested endpoint	You need zero help to build a new module from scratch

18. Final Mindset — Think Like an Engineer

You Are Building a System, Not Just Code

Every line you write has downstream consequences. Ask these questions before coding:

WHAT HAPPENS WHEN THIS FAILS?

A missing await silently breaks an order. A missing transaction corrupts stock.
Write code that fails loudly and clearly, not silently.

WHO IS AFFECTED IF THIS DATA IS WRONG?

In an ERP, a pricing bug affects every sale. A stock bug affects every order.
Think about consequences before writing.

WHAT DOES THE NEXT DEVELOPER NEED TO UNDERSTAND THIS?

Write code for the developer who comes after you, not for the computer.

Clear variable names, clear function names, and clear error messages cost nothing.

COULD TWO USERS BREAK THIS SIMULTANEOUSLY?

Any operation that reads then writes requires a transaction or atomic update.

Always think: what happens with two concurrent requests?